

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №5-6

Выполнил:
Студент группы ИУ5-35Б
Эльсон Э.К.

Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Шаблоны проектирования и модульное тестирование в Python.

Цель лабораторной работы: изучение реализации шаблонов проектирования и возможностей модульного тестирования в языке Python.

Требования к отчету:

Отчет по лабораторной работе должен содержать:

1. титульный лист;
2. описание задания;
3. текст программы;
4. экранные формы с примерами выполнения программы.

Задание:

1. Необходимо для произвольной предметной области реализовать от одного до трех шаблонов проектирования: один порождающий, один структурный и один поведенческий. В качестве справочника шаблонов можно использовать [следующий каталог](#). Для сдачи лабораторной работы в минимальном варианте достаточно реализовать один паттерн.
2. В модульных тестах необходимо применить следующие технологии:
 - o TDD - фреймворк.
 - o BDD - фреймворк.
 - o Создание Mock-объектов.

Листинг кода

app.py

```
"""
```

Демонстрация работы всех трех паттернов

```
"""
```

```
from patterns import (  
    DatabaseConnection,  
    OldSystemAdapter, ModernSystem,  
    NewsPublisher, EmailSubscriber, SMSSubscriber  
)
```

```
def demo_singleton():  
    print("=== SINGLETON ===")  
    db1 = DatabaseConnection()  
    db2 = DatabaseConnection()
```

```
print(f"Same instance: {db1 is db2}")
print(f"DB1: {db1.connect()}")
print(f"DB2 query: {db2.query('SELECT * FROM users')}")
print()
```

```
def demo_adapter():
    print("=== ADAPTER ===")
    adapter = OldSystemAdapter()
    modern = ModernSystem(adapter)
    result = modern.make_request()
    print(f"Result: {result}")
    print()
```

```
def demo_observer():
    print("=== OBSERVER ===")
    publisher = NewsPublisher()

    email_sub = EmailSubscriber("user@example.com")
    sms_sub = SMSSubscriber("+79001234567")

    publisher.subscribe(email_sub)
    publisher.subscribe(sms_sub)

    publisher.publish_news("Python 3.12 released!")

    publisher.unsubscribe(email_sub)
    publisher.publish_news("AI conference starts tomorrow")
    print()
```

```

if __name__ == "__main__":
    demo_singleton()
    demo_adapter()
    demo_observer()
    print("All patterns demonstrated!")

```

patterns.py

"""

Все три шаблона проектирования в одном файле:

1. Singleton (Порождающий)
2. Adapter (Структурный)
3. Observer (Поведенческий)

"""

===== 1. SINGLETON (Порождающий) =====

class DatabaseConnection:

 _instance = None

 def __new__(cls):

 if cls._instance is None:

 cls._instance = super().__new__(cls)

 return cls._instance

 def connect(self):

 return "Database connected"

 def query(self, sql):

 return f"Executed: {sql}"

===== 2. ADAPTER (Структурный) =====

```
class OldSystem:
    def legacy_request(self):
        return "Old system response"
```

```
class ModernSystem:
    def __init__(self, adapter):
        self.adapter = adapter

    def make_request(self):
        return self.adapter.request()
```

```
class OldSystemAdapter:
    def __init__(self):
        self.old_system = OldSystem()

    def request(self):
        old_response = self.old_system.legacy_request()
        return f"Adapted: {old_response}"
```

===== 3. OBSERVER (Поведенческий) =====

```
class NewsPublisher:
    def __init__(self):
        self._subscribers = []

    def subscribe(self, subscriber):
        self._subscribers.append(subscriber)

    def unsubscribe(self, subscriber):
        self._subscribers.remove(subscriber)
```

```
def publish_news(self, news):  
    print(f" Publishing: {news}")  
    for subscriber in self._subscribers:  
        subscriber.update(news)
```

```
class EmailSubscriber:
```

```
    def __init__(self, email):  
        self.email = email
```

```
    def update(self, news):  
        print(f" Email to {self.email}: {news}")
```

```
class SMSSubscriber:
```

```
    def __init__(self, phone):  
        self.phone = phone
```

```
    def update(self, news):  
        print(f" SMS to {self.phone}: {news}")
```

observer.feature

Feature: Observer Pattern

As a news system developer

I want to use Observer pattern

To notify subscribers about news

Scenario: Subscribe to news

Given a news publisher is created

And an email subscriber with address "user@example.com" is created

When the subscriber subscribes to the publisher

Then the subscriber should be in publisher's subscribers list

Scenario: Receive news notification

Given a news publisher is created

And an email subscriber with address "user@example.com" is created

And the subscriber subscribes to the publisher

When the publisher publishes news "Important update"

Then the subscriber should receive notification with "Important update"

test_mock.py

"""

Тесты с использованием Mock-объектов

"""

from unittest.mock import Mock, patch

from patterns import DatabaseConnection, NewsPublisher

def test_singleton_with_mock():

db = DatabaseConnection()

with patch.object(db, 'connect', return_value="Mocked Connection"):

result = db.connect()

assert result == "Mocked Connection"

db.connect.assert_called_once()

def test_observer_with_mocks():

publisher = NewsPublisher()

mock_subscriber = Mock()

```
publisher.subscribe(mock_subscriber)
publisher.publish_news("Mocked news")
```

```
mock_subscriber.update.assert_called_once_with("Mocked news")
```

test_patterns.py

```
"""
```

TDD-тесты для всех трех паттернов

```
"""
```

```
import pytest
```

```
from patterns import (
```

```
    DatabaseConnection,
```

```
    OldSystemAdapter, ModernSystem,
```

```
    NewsPublisher, EmailSubscriber, SMSSubscriber
```

```
)
```

```
# === TESTS FOR SINGLETON ===
```

```
def test_singleton_instance():
```

```
    db1 = DatabaseConnection()
```

```
    db2 = DatabaseConnection()
```

```
    assert db1 is db2
```

```
def test_singleton_methods():
```

```
    db = DatabaseConnection()
```

```
    assert db.connect() == "Database connected"
```

```
    assert "SELECT" in db.query("SELECT * FROM users")
```

```
# === TESTS FOR ADAPTER ===
```

```
def test_adapter():
```



```

adapter = OldSystemAdapter()
result = adapter.request()
assert "Adapted: Old system response" in result

def test_modern_system_with_adapter():
    adapter = OldSystemAdapter()
    modern = ModernSystem(adapter)
    assert "Adapted:" in modern.make_request()

# === TESTS FOR OBSERVER ===

def test_observer_subscribe():
    publisher = NewsPublisher()
    subscriber = EmailSubscriber("test@test.com")
    publisher.subscribe(subscriber)
    assert subscriber in publisher._subscribers

def test_observer_publish(capsys):
    publisher = NewsPublisher()
    subscriber = EmailSubscriber("test@test.com")
    publisher.subscribe(subscriber)

    publisher.publish_news("Test news")

    captured = capsys.readouterr()
    assert "Test news" in captured.out
    assert "test@test.com" in captured.out

```

Результат выполнения программы:

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab5-6> py app.py
=== SINGLETON ===
Same instance: True
DB1: Database connected
DB2 query: Executed: SELECT * FROM users
```

```
=== ADAPTER ===
Result: Adapted: Old system response
```

```
=== OBSERVER ===
Publishing: Python 3.12 released!
Email to user@example.com: Python 3.12 released!
SMS to +79001234567: Python 3.12 released!
Publishing: AI conference starts tomorrow
SMS to +79001234567: AI conference starts tomorrow
```

All patterns demonstrated!

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab5-6> pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-7.4.0, pluggy-1.6.0 -- C:\Users\elson\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab5-6
plugins: bdd-6.1.1, mock-3.11.1
collected 8 items

tests/test_mocks.py::test_singleton_with_mock PASSED [ 12%]
tests/test_mocks.py::test_observer_with_mocks PASSED [ 25%]
tests/test_patterns.py::test_singleton_instance PASSED [ 37%]
tests/test_patterns.py::test_singleton_methods PASSED [ 50%]
tests/test_patterns.py::test_adapter PASSED [ 62%]
tests/test_patterns.py::test_modern_system_with_adapter PASSED [ 75%]
tests/test_patterns.py::test_observer_subscribe PASSED [ 87%]
tests/test_patterns.py::test_observer_publish PASSED [100%]

===== 8 passed in 0.06s =====
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab5-6> cat tests/observer.feature
```

Feature: Observer Pattern

As a news system developer
I want to use Observer pattern
To notify subscribers about news

Scenario: Subscribe to news

Given a news publisher is created
And an email subscriber with address "user@example.com" is created
When the subscriber subscribes to the publisher
Then the subscriber should be in publisher's subscribers list

Scenario: Receive news notification

Given a news publisher is created
And an email subscriber with address "user@example.com" is created
And the subscriber subscribes to the publisher
When the publisher publishes news "Important update"
Then the subscriber should receive notification with "Important update"

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab5-6> sls "Mock|patch" tests/test_mocks.py
```

```
tests\test_mocks.py:2:Тесты с использованием Mock-объектов
tests\test_mocks.py:4:from unittest.mock import Mock, patch
tests\test_mocks.py:7:def test_singleton_with_mock():
tests\test_mocks.py:10:     with patch.object(db, 'connect', return_value="Mocked Connection"):
tests\test_mocks.py:12:         assert result == "Mocked Connection"
tests\test_mocks.py:15:def test_observer_with_mocks():
tests\test_mocks.py:17:     mock_subscriber = Mock()
tests\test_mocks.py:19:     publisher.subscribe(mock_subscriber)
tests\test_mocks.py:20:     publisher.publish_news("Mocked news")
tests\test_mocks.py:22:     mock_subscriber.update.assert_called_once_with("Mocked news")
```